Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Факультет электронно-информационных систем

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Введение в нейронные сети: построение многослойного перцептрона

Специальность АС-68

Выполнил:
Студент 3-го курса
Группы АС-68
Богданов В. С.

Проверил:
А. А. Крощенко
Доцент кафедры ИИТ,
«___»_____ 2026 г.

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации.

Общее задание:

- импортировать библиотеки, загрузить датасет, выполнить стандартизацию и кодирование признаков;
- разделить данные на обучающую и тестовую выборки и преобразовать их в тензоры PyTorch;
- определить архитектуру сети как класс, наследуемый от torch.nn.Module;
- инициализировать модель, функцию потерь и оптимизатор;
- написать цикл обучения (forward pass, расчёт потерь, обнуление градиентов, обратное распространение, шаг оптимизации);
- оценить модель на тестовой выборке с помощью метрик accuracy, F1-score и др.

Вариант 1

Классификация ирисов (Iris)

- Iris
- Задача: определить вид ириса (3 класса).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 10 нейронами (ReLU);
  - выходной слой с 3 нейронами (Softmax).
- **Эксперимент:** добавить второй скрытый слой с 5 нейронами и сравнить точность.

Код программы:

```
# Лабораторная работа №4. Вариант 1: классификация ирисов (MLP, PyTorch)
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix,
classification_report

HERE = Path(__file__).resolve().parent   # папка со скриптом: графики сохраняем сюда



def find_csv(name="iris.csv"):
    """Ищем iris.csv в папке скрипта, затем в родительских папках."""
    for d in [HERE, *HERE.parents[:4]]:
        if (d / name).exists():
            return d / name
    raise FileNotFoundError(f"{name} не найден рядом со скриптом или в родительских
папках ({HERE})")



SEED = 42
EPOCHS = 500
LR = 0.001



def set_seed(seed):
    torch.manual_seed(seed)
    np.random.seed(seed)



# 1. Импорт и подготовка данных
df = pd.read_csv(find_csv())
X = df.drop(columns="variety").values
y = LabelEncoder().fit_transform(df["variety"])       # Setosa/Versicolor/Virginica ->
0/1/2
class_names = sorted(df["variety"].unique())

# То же разбиение, что и в ЛР№3: 70/30 со стратификацией
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=SEED, stratify=y)

# Стандартизация: параметры считаем только по train
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Тензоры PyTorch
X_train_t = torch.tensor(X_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.long)    # для CrossEntropyLoss метки -
long
y_test_t = torch.tensor(y_test, dtype=torch.long)
print("train:", X_train_t.shape, "test:", X_test_t.shape, "\n")



# 2. Архитектуры
class MLP(nn.Module):
    """Основная: 4 -> 10 (ReLU) -> 3 (Softmax)."""
    def __init__(self, n_in=4, n_hidden=10, n_out=3):
        super().__init__()
        self.fc1 = nn.Linear(n_in, n_hidden)
        self.relu = nn.ReLU()
        self.out = nn.Linear(n_hidden, n_out)

    def forward(self, x):
        # Возвращаем логиты: Softmax встроен в nn.CrossEntropyLoss,
        # вероятности получаем отдельно через torch.softmax при предсказании
        return self.out(self.relu(self.fc1(x)))
```

```
class MLP2(nn.Module):
    """Эксперимент: 4 -> 10 (ReLU) -> 5 (ReLU) -> 3 (Softmax)."""
    def __init__(self, n_in=4, n_h1=10, n_h2=5, n_out=3):
        super().__init__()
        self.fc1 = nn.Linear(n_in, n_h1)
        self.fc2 = nn.Linear(n_h1, n_h2)
        self.relu = nn.ReLU()
        self.out = nn.Linear(n_h2, n_out)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        return self.out(x)



# 3-4. Обучение
def train_model(model_cls, seed=SEED, epochs=EPOCHS, verbose=False):
    set_seed(seed)
    model = model_cls()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LR)
    train_losses, test_losses = [], []

    for epoch in range(1, epochs + 1):
        model.train()
        y_pred = model(X_train_t)                # forward pass
        loss = criterion(y_pred, y_train_t)      # потери
        optimizer.zero_grad()                    # обнуление градиентов
        loss.backward()                          # обратное распространение
        optimizer.step()                         # шаг оптимизации

        model.eval()
        with torch.no_grad():
            test_loss = criterion(model(X_test_t), y_test_t).item()
        train_losses.append(loss.item())
        test_losses.append(test_loss)

        if verbose and (epoch % 50 == 0 or epoch == 1):
            print(f"  epoch {epoch:4d}: train loss = {loss.item():.4f}, test loss =
{test_loss:.4f}")
    return model, train_losses, test_losses



# 5. Оценка
def evaluate(model):
    model.eval()
    with torch.no_grad():
        logits = model(X_test_t)
        probs = torch.softmax(logits, dim=1)
        pred = torch.argmax(probs, dim=1).numpy()
    return pred, probs.numpy()



def report(name, model):
    pred, _ = evaluate(model)
    acc = accuracy_score(y_test, pred)
    f1 = f1_score(y_test, pred, average="macro")
    print("=" * 60)
    print(f"{name}: accuracy = {acc:.4f}, macro F1 = {f1:.4f}")
    print("Матрица ошибок:\n", confusion_matrix(y_test, pred))
    print(classification_report(y_test, pred, target_names=class_names, digits=3))
    n_params = sum(p.numel() for p in model.parameters())
    print("Число параметров:", n_params, "\n")
```

```
    return acc, f1


print("Обучение модели 1: 4-10-3")
model1, tr1, te1 = train_model(MLP, verbose=True)
acc1, f1_1 = report("MLP (1 скрытый слой, 10 нейронов)", model1)


print("Обучение модели 2: 4-10-5-3")
model2, tr2, te2 = train_model(MLP2, verbose=True)
acc2, f1_2 = report("MLP (2 скрытых слоя: 10 и 5 нейронов)", model2)


# Тест из 45 объектов мал, поэтому повторяем обучение с разными seed
print("Устойчивость: 10 запусков с разной инициализацией весов")
seeds = range(10)
accs1, accs2 = [], []
for s in seeds:
    m1, _, _ = train_model(MLP, seed=s)
    m2, _, _ = train_model(MLP2, seed=s)
    accs1.append(accuracy_score(y_test, evaluate(m1)[0]))
    accs2.append(accuracy_score(y_test, evaluate(m2)[0]))
print(f"  4-10-3   : accuracy = {np.mean(accs1):.4f} ± {np.std(accs1):.4f}")
print(f"  4-10-5-3 : accuracy = {np.mean(accs2):.4f} ± {np.std(accs2):.4f}\n")


# Графики
fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
for ax, (tr, te, title) in zip(axes, [(tr1, te1, "4-10-3"), (tr2, te2, "4-10-5-3")]):
    ax.plot(tr, label="train")
    ax.plot(te, label="test", ls="--")
    ax.set_title(f"Функция потерь: {title}")
    ax.set_xlabel("Эпоха"); ax.grid(alpha=.3); ax.legend()
axes[0].set_ylabel("CrossEntropy loss")
plt.tight_layout(); plt.savefig(HERE / "mlp_loss.png", dpi=150); plt.close()


plt.figure(figsize=(6, 4))
names = ["1 скрытый\n(10)", "2 скрытых\n(10, 5)"]
bars = plt.bar(names, [acc1, acc2], color=["#4c72b0", "#55a868"])
plt.ylim(0.8, 1.02); plt.ylabel("Accuracy (test)"); plt.title("Сравнение архитектур
MLP")
for b, v in zip(bars, [acc1, acc2]):
    plt.text(b.get_x() + b.get_width() / 2, v + .005, f"{v:.3f}", ha="center")
plt.tight_layout(); plt.savefig(HERE / "mlp_compare.png", dpi=150); plt.close()
```

Результаты работы программы:

1. Данные. Набор Iris содержит 150 объектов, 4 числовых признака (sepal.length, sepal.width, petal.length, petal.width) и целевую переменную variety с тремя классами (Setosa, Versicolor, Virginica). Данные разделены в соотношении 70/30 со стратификацией по классам (random_state = 42, как в лабораторной работе №3): 105 объектов в обучающей и 45 в тестовой выборке. Признаки стандартизированы с помощью StandardScaler, параметры которого вычислены только по обучающей выборке. Метки классов закодированы числами 0, 1, 2 (LabelEncoder). Признаки переведены в тензоры float32, метки — в тензоры long, как того требует CrossEntropyLoss.

2. Архитектура и обучение. Базовая модель MLP имеет структуру 4 → 10 (ReLU) → 3 и содержит 83 обучаемых параметра. Метод forward возвращает логиты: функция Softmax встроена в nn.CrossEntropyLoss, а вероятности классов вычисляются отдельно через torch.softmax на этапе оценки. Использованы оптимизатор Adam (lr = 0,001) и обучение на всей обучающей выборке целиком (полный батч) в течение 500 эпох. Потери на обучающей выборке снизились с 1,0435 до 0,1765, на тестовой — с 1,0457 до 0,2652 (рис. 1). Разрыв между train и test небольшой, признаков переобучения нет; кривые к 500-й эпохе ещё продолжают снижаться.

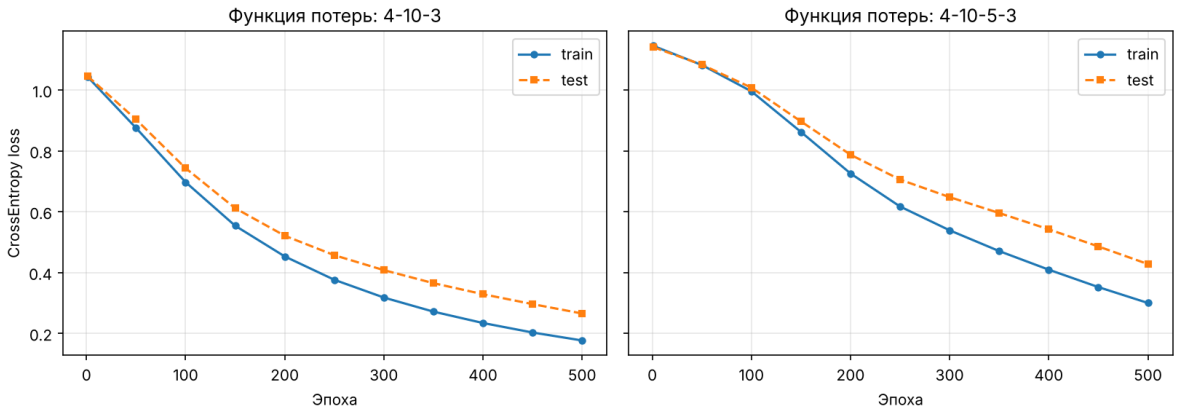


Рис. 1 – Динамика функции потерь при обучении моделей (значения приведены каждые 50 эпох)

3. Оценка базовой модели (4-10-3). На тестовой выборке получены accuracy = 0,9111 и macro F1 = 0,9107. Setosa классифицируется безошибочно. Ошибки возникают только между Versicolor и Virginica, которые в пространстве признаков частично перекрываются: один объект Versicolor принят за Virginica и три объекта Virginica — за Versicolor (4 ошибки из 45).

Таблица 1 – Матрица ошибок модели 4-10-3 (строки — истинный класс, столбцы — предсказанный)

	Setosa	**Versicolor**	**Virginica**
Setosa	15	0	0
Versicolor	0	14	1
Virginica	0	3	12

4. Эксперимент: добавление второго скрытого слоя. Вторая модель имеет структуру 4 → 10 (ReLU) → 5 (ReLU) → 3 (123 параметра), остальные условия обучения не менялись. Потери на обучающей выборке снизились с 1,1449 до 0,2998, на тестовой — до 0,4277, то есть при тех же 500 эпохах более глубокая сеть обучается медленнее и к концу обучения не успевает достичь уровня базовой модели (рис. 1). На тестовой выборке получены accuracy = 0,8444 и macro F1 = 0,8443: допущено 7 ошибок вместо 4, при этом дополнительные ошибки пришлись на класс Versicolor (4 объекта предсказаны как Virginica).

Таблица 2 – Матрица ошибок модели 4-10-5-3

	Setosa	**Versicolor**	**Virginica**
Setosa	15	0	0
Versicolor	0	11	4
Virginica	0	3	12

Таблица 3 – Метрики по классам на тестовой выборке (P — precision, R — recall)

Класс	**4-10-3 P**	**4-10-3 R**	**4-10-3 F1**	**4-10-5-3 P**	**4-10-5-3 R**	**4-10-5-3 F1**
Setosa	1,000	1,000	1,000	1,000	1,000	1,000
Versicolor	0,824	0,933	0,875	0,786	0,733	0,759
Virginica	0,923	0,800	0,857	0,750	0,800	0,774

Таблица 4 – Сравнение архитектур (одно обучение, random seed = 42)

Архитектура	**Параметры**	**Loss (train)**	**Loss (test)**	**Accuracy**	**Macro F1**	**Ошибок**
4-10-3	83	0,1765	0,2652	0,9111	0,9107	4
4-10-5-3	123	0,2998	0,4277	0,8444	0,8443	7

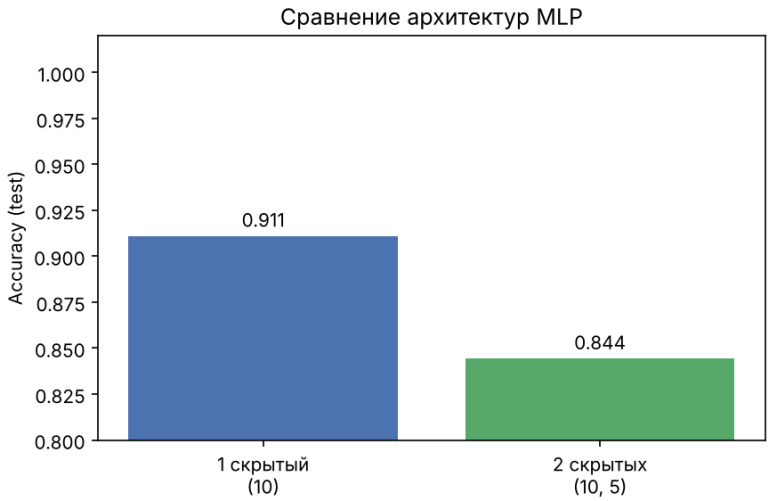


Рис. 2 – Сравнение точности моделей на тестовой выборке

5. Устойчивость результата. Тестовая выборка состоит всего из 45 объектов, поэтому один неверно классифицированный объект меняет accuracy на 2,2 процентного пункта. Кроме того, результат зависит от случайной инициализации весов. Для проверки каждая архитектура была обучена 10 раз с разными начальными весами (seed от 0 до 9).

Таблица 5 – Accuracy на тестовой выборке по 10 запускам

Архитектура	**Среднее**	**Стандартное отклонение**
4-10-3	0,8911	0,0210
4-10-5-3	0,9111	0,0263

Усреднённый результат противоположен единичному запуску: в среднем модель с двумя скрытыми слоями оказалась точнее (0,911 против 0,891), хотя при seed = 42 она проиграла (0,844 против 0,911). Разница средних (0,02) сопоставима со стандартным отклонением (0,02–0,03), поэтому статистически значимого преимущества ни у одной архитектуры нет. Результаты также ниже, чем у классических методов из лабораторной работы №3 (k-NN — 0,956, Decision Tree — 0,978, SVM — 0,933). Одна из вероятных причин — недообучение: потери к 500-й эпохе всё ещё снижались, а при lr = 0,001 и полнобатчевом обучении на 105 объектах это всего 500 шагов оптимизации. Это предположение в работе не проверялось.

Вывод: на практике построил, обучил и оценил многослойный перцептрон для классификации ирисов с помощью PyTorch. Освоил основные этапы: подготовку данных и преобразование в тензоры, описание архитектуры через nn.Module, выбор функции потерь (CrossEntropyLoss для многоклассовой задачи) и оптимизатора

(Adam), написание цикла обучения и оценку модели в режиме eval с отключением градиентов. На тестовой выборке базовая модель 4-10-3 показала accuracy = 0,911 и macro F1 = 0,911, Setosa определяется безошибочно, ошибки возникают между Versicolor и Virginica. Добавление второго скрытого слоя с 5 нейронами при единичном обучении снизило точность до 0,844, однако по 10 запускам с разной инициализацией средняя точность составила 0,891 ± 0,021 для базовой модели и 0,911 ± 0,026 для модели с двумя слоями, то есть различие находится в пределах разброса и не позволяет утверждать, что более глубокая архитектура лучше или хуже. Для столь простой задачи с хорошо разделимыми классами достаточно одного скрытого слоя; для более точного сравнения архитектур необходимо большее число эпох и оценка на кросс-валидации.